

15주차 예습과제

≡ 범위 자연어처리와 컴퓨터비전 심층학습 7장 (BERT)

BERT

BERT

: 트랜스포머 기반 양방향 인코더를 사용하는 자연어 처리 모델

- **양방향 인코더**: 입력 시퀀스를 양쪽 방향에서 처리하여 이전과 이후의 단어를 모두 참조하면서 단어의 의미와 문맥을 파악한다.
- BERT는 대규모 데이터를 이용해 사전 학습 되어 있어 전이 학습에 주로 활용됨
- 일부 또는 전체를 다른 작업에서 재사용해 적은 양의 데이터로도 높은 정확도 달성 가능, 모델 학습 시간이 크게 단축됨.

- [BERT 모델과 트랜스포머 비교]

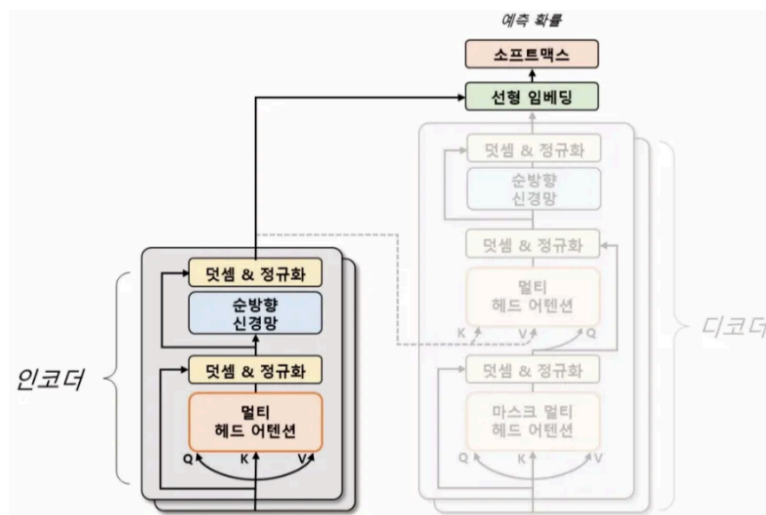


그림 7.13 트랜스포머와 BERT 모델 비교

⇒ BERT는 트랜스포머의 인코더 모듈만을 사용해 입력 문장을 처리함.

- 인코더: 입력 문장의 단어들을 임베딩해 각 단어의 의미를 벡터화 → 순차적으로 처리하여 문장 전체의 의미를 추출함.
- BERT는 입력 문장의 단어를 좌우 양방향으로 처리해 문맥 정보를 모델링하므로 인코더 계층만을 사용해 학습한다.

사전 학습 방법

- **마스킹된 언어 모델링 (MLM):** 입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어를 예측하는 방식이다. 이 과정에서 양방향 문맥 정보를 참고한다.
 - 'I'm learning PyTorch' → 여기서 'learning'을 마스킹하면 'I'm [MASK] PyTorch'가 되어 BERT는 [MASK]에 들어갈 단어를 예측한다.
 - 이 과정에서 양방향 문맥 정보를 참고한다.
- **다음 문장 예측 (NSP):** 두 개의 문장이 주어졌을 때, 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지 여부를 판단하는 작업이다. 이 과정에서 두 문장 간의 관계를 학습한다.
- **특수 토큰:** 입력문장에 특수한 토큰들을 추가해 모델이 학습하고 추론하는 과정에서 필요한 정보를 제공함.
 - **[CLS]:** 토큰은 입력 문장의 시작 부분에 추가되어 문장 분류 작업을 위한 정보를 제공한다
 - **[SEP]:** 토큰은 입력 문장 내에서 두 개 이상의 문장을 구분하기 위해 사용된다
 - **[MASK]:** 토큰은 입력 문장 내에서 임의로 선택된 단어를 가리키며, 마스킹된 실제 단어를 예측하는 데 사용된다

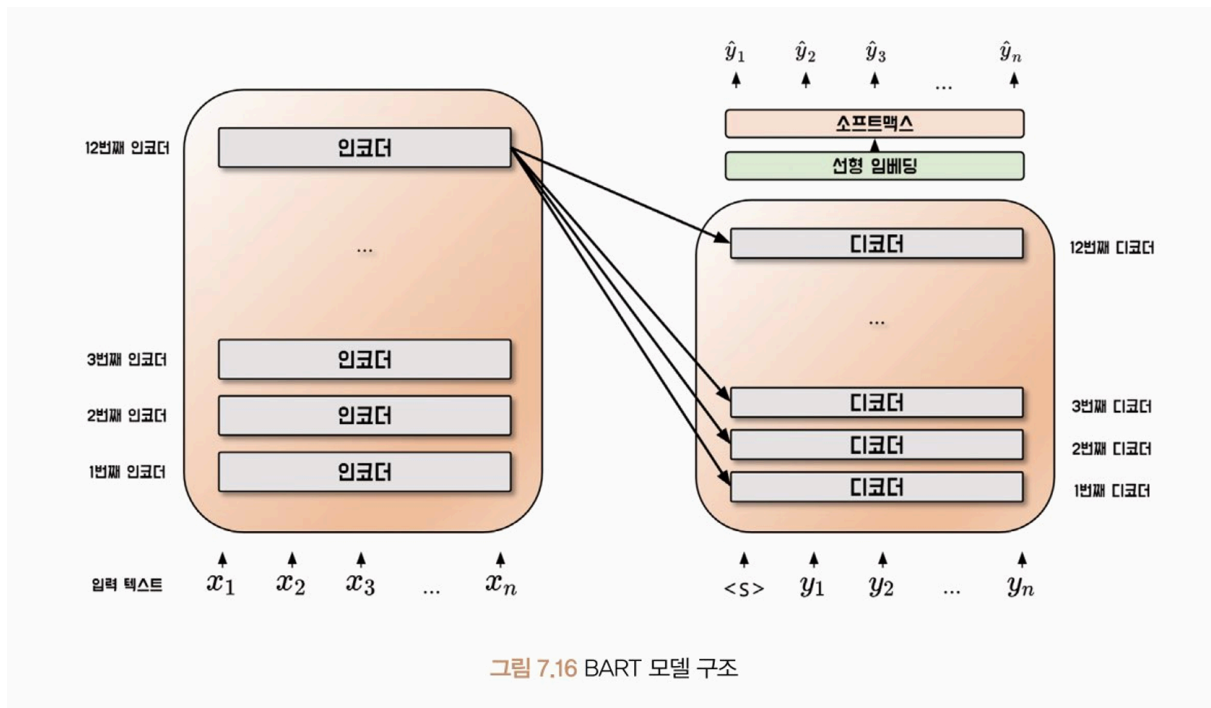
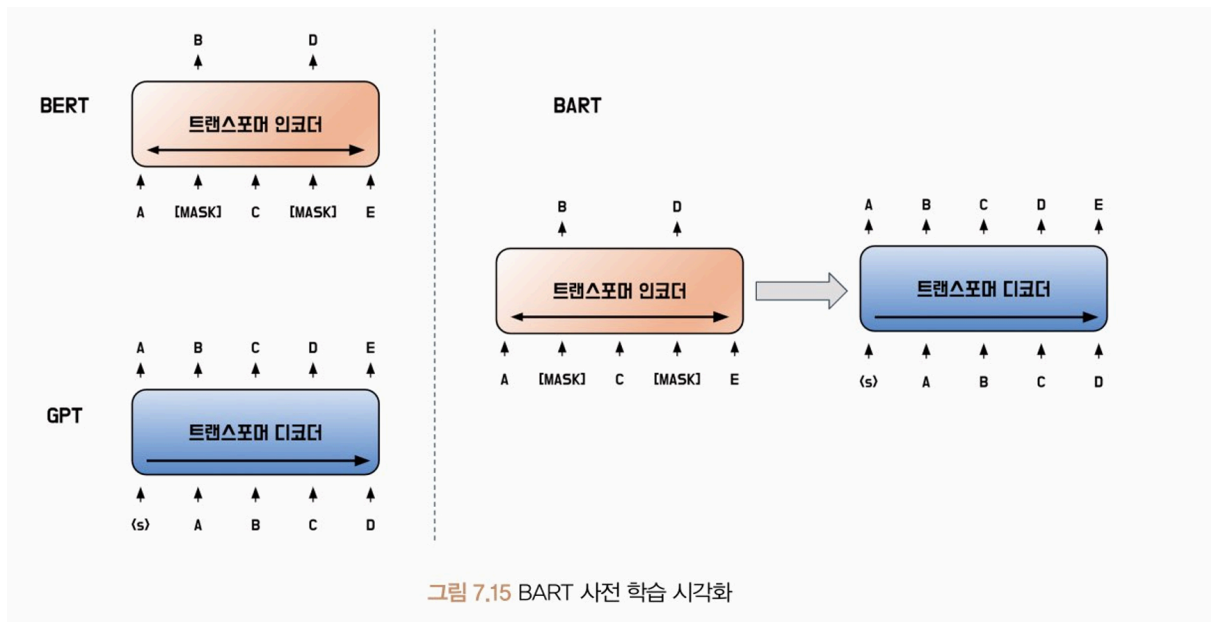
⇒ BERT 모델의 입력 시퀀스: [CLS] 문장1 [SEP] 문장2 [MASK] [SEP] 이런 구조가 될 수 있음.



- BERT는 사전 학습을 수행한 후, 미세 조정 기법을 통해 다양한 자연어 처리 작업에 적용할 수 있다.
 - 예: 문장 분류, 감성 분석, 질문 응답, 기계 번역 등의 다양한 작업
 - 미세 조정 과정에서는 해당 작업에 맞는 계층을 추가하고 손실 함수를 정의해 학습함.
- 분류 문제에서는 일반적으로 [CLS] 토큰 벡터를 사용하고 각 토큰마다 예측이 필요한 경우에는 모든 토큰 벡터를 사용함.

BART

- BART는 2019년 메타의 FAIR 연구소에서 발표한 트랜스포머 기반의 모델이다.
- BERT의 인코더와 GPT의 디코더를 결합한 시퀀스-시퀀스(Seq2Seq) 구조로, 노이즈 제거 오토인코더로 사전 학습된다.
 - **오토인코더 사전 학습 방법:** 입력 문장에 임의로 노이즈를 추가하고 원래 문장을 복원하도록 학습
 - **BERT와 GPT의 장점 결합:** 인코더를 사용함으로써 양방향 문맥 정보를 반영하고, 디코더를 사용함으로써 문장 생성 분야에서 BERT의 단점을 개선한다.



- **BART:** 인코더의 **마지막 계층** 과 디코더의 **각 계층** 사이에만 어텐션 연산을 수행한다.
- **트랜스포머:** 인코더의 **모든 계층** 과 디코더의 **모든 계층** 사이의 어텐션 연산을 수행한다.

[BART의 인코더]

- BART의 인코더는 입력 문장의 각 단어를 임베딩하고 여러 층의 인코더를 거친다.
- **마지막 계층**에서는 입력 문장 전체의 의미를 가장 잘 반영하는 벡터가 생성된다.

[BART의 디코더]

- 인코더에서 생성된 벡터는 디코더가 출력 문장을 생성할 때 참고된다.
- 디코더의 각 계층에서는 이 벡터와 이전 계층에서 생성된 출력 문장의 정보를 활용해 출력 문장을 생성한다.
- BART에서는 인코더의 마지막 계층과 디코더의 각 계층 사이에서만 어텐션 연산을 수행하므로 정보 전달을 최적화하고 메모리 사용량을 줄일 수 있다.

사전 학습 방법

- BART의 인코더는 BERT의 마스킹된 언어 모델링(MLM) 이외에도 다양한 노이즈 기법을 사용한다
- 문장의 일부를 가리는 토큰 마스킹 기법 이외에도 토큰 삭제, 문장 교환, 문서 회전, 텍스트 채우기 기법이 사용된다.



1. 토큰 마스킹 (Token Masking)

- 토큰 마스킹은 BERT에서 사용한 MLM과 **동일한 기법**으로, 입력 문장의 일부 토큰을 **마스킹 토큰으로 치환**하는 방법이다.
- 이 기법을 이용하면 문장 내에서 어떤 단어가 중요한 역할을 하는지를 학습할 수 있으며, 입력 문장의 문맥을 이해하고 문장 내에서 중요한 정보를 추출하는 능력이 향상된다.

2. 토큰 삭제 (Token Deletion)

- 토큰 삭제는 입력 문장의 일부 토큰을 치환하는 것이 아니라, **삭제하는 방법**이다.
- 토큰 마스킹과 다르게 어떤 위치의 토큰이 삭제되었는지도 맞춰야 한다. 이 기법을 이용하면 입력 문장에서 불필요한 정보나 중요하지 않은 정보를 자동으로 필터링해 처리할 수 있게 되며,
- 모델의 학습과 예측 시간을 줄이고, 모델의 일반화 성능을 향상시킬 수 있다.

3. 문장 순열 (Sentence Permutation)

- 문장 순열은 마침표(.)를 기준으로 문장을 나눈 뒤, **문장의 순서를 섞는 방법**이다. 모델은 **원래의 문장 순서를 맞춰야** 한다.
- 이 기법은 문장 내에서 단어들이 어떻게 연결되어 있는지 더 잘 파악하게 되며, 입력 문장이 다른 순서로 주어졌을 때에도 잘 처리할 수 있게 된다.

4. 문서 회전 (Document Rotation)

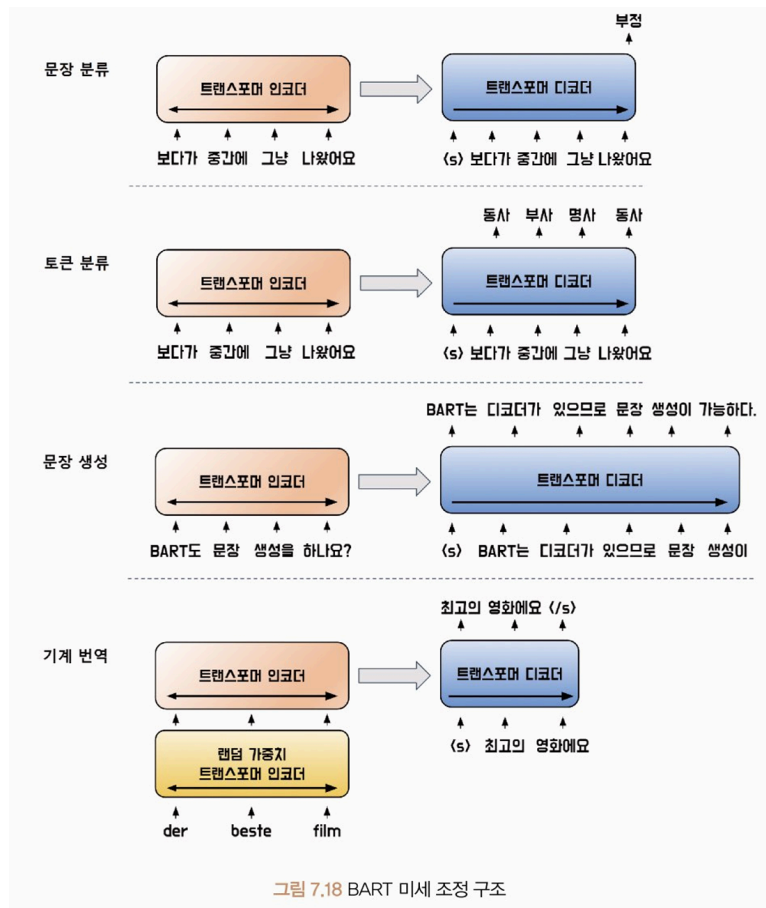
- 문서 회전은 **임의의 토큰으로 문서가 시작하도록** 한다.
- 문장 순열과는 다르게 문장의 순서는 유지한다. 모델은 **문서의 원래의 시작 토큰을 맞춰야** 한다. 이 기법은 모델이 문서의 시작점을 인식할 수 있도록 한다.

5. 텍스트 채우기 (Text Infilling)

- 텍스트 채우기는 몇 개의 토큰을 하나의 구간으로 묶고 **일부 구간을 마스크 토큰으로 대체**한다. 모델은 **연속된 마스크 토큰을 복구**하되, 실제로는 마스킹되지 않은 토큰도 구분해야 한다.
- 이를 통해 모델이 누락된 단어를 예측하게 유도함으로써 더 많은 정보를 활용하여 입력 문장을 더 잘 이해할 수 있게 된다.

미세 조정 방법

- BART는 인코더와 디코더를 모듈 사용하는 구조를 가지고 있기 때문에 미세 조정 시 각 다운스트림 작업에 맞게 입력 문장을 구성해야 한다.
- 즉, 인코더와 디코더에 다른 문장 구조로 입력한다.



- 문장 분류 작업: 입력 문장을 인코더와 디코더 동일하게 입력
 - 디코더의 마지막 토큰 은닉 상태를 선형 분류기의 입력값으로 사용
- 토큰 분류 작업: 입력 문장을 인코더와 디코더 동일하게 입력
 - 디코더의 각 시점별 마지막 토큰 은닉 상태를 선형 분류기의 입력값으로 사용
- BART는 트랜스포머 디코더를 사용하기 때문에 BERT가 해결하지 못했던 문장 생성 작업을 수행할 수 있다.
- 특히 입력값을 조작해 출력을 생성하는 **추상적 질의응답**과 **문장 요약**과 같은 작업에 적합하다.
- BART는 사전학습 방식이 BERT와 유사해 문장 의미를 깊이 이해하고 이를 바탕으로 새로운 문장을 생성할 수 있다.
- 예를 들어, 기계 번역에서는 사전학습된 인코더에 번역용 인코더를 추가하여 학습할 수 있다.
추가 인코더는 기존 단어 사전을 사용하지 않고, 디코더는 사전 학습된 가중치와 단어 사전을 그대로 활용한다.

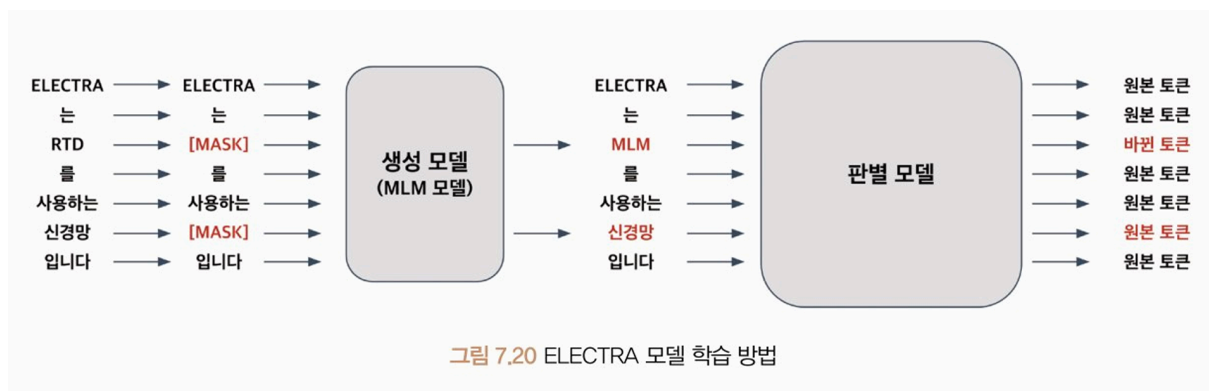
1. 첫 단계: 새로 추가된 인코더의 가중치와 위치 임베딩, 첫 인코더 층의 셀프 어텐션 입력 행렬 가중치만 학습
2. 두 번째 단계: 모든 신경망 가중치를 작은 학습률로 반복 학습

⇒ BART는 문장 내부의 토큰 간 상관관계를 정교하게 파악하여 문장의 의미를 더 정확히 이해 가능

ELECTRA

- 입력을 마스킹하는 대신 생성자와 판별자를 사용해 사전학습을 수행하는 트랜스포머 기반의 모델
- 사전 학습 방식 : 생성자와 판별자를 학습 → 생성적 적대 신경망 GAN과 유사한 방법으로 학습 수행
- BERT보다 매개 변수 수가 적음 → 모델의 크기가 줄어들어 모델을 더 빠르게 실행할 수 있으며, 더 적은 메모리를 사용함.

사전 학습 방법

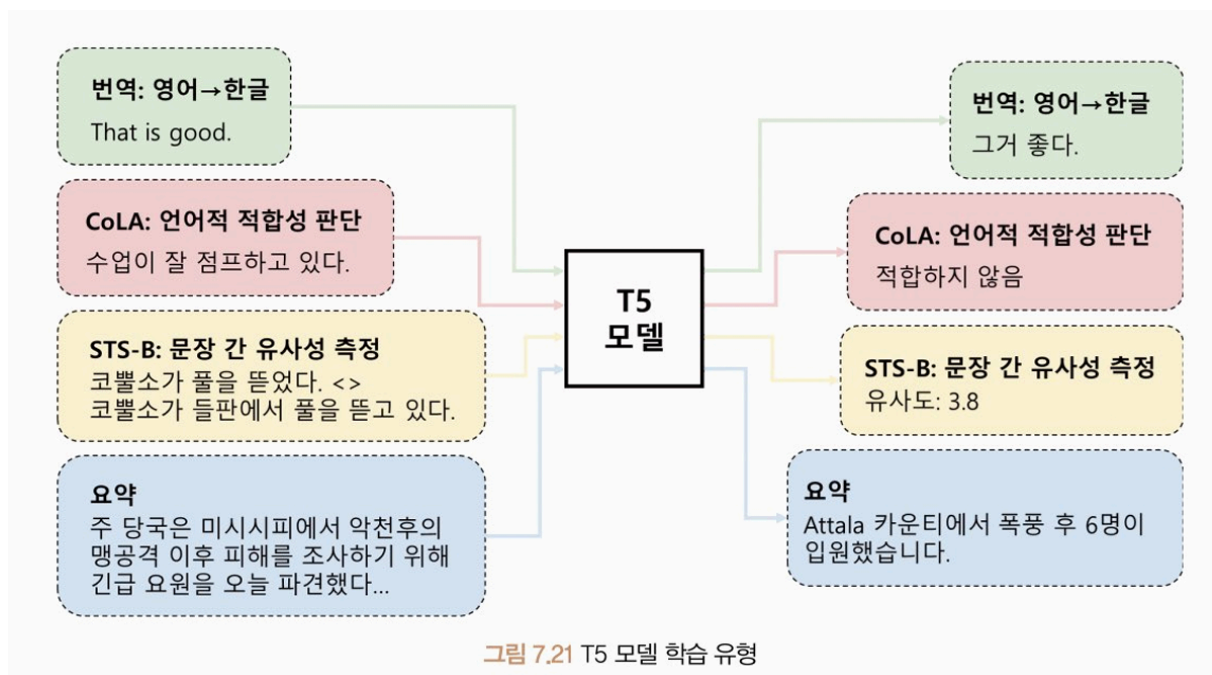


- 생성자 모델과 판별자 모델을 모두 트랜스포머 인코더 구조를 따름
- 생성자 모델은 BERT의 마스킹된 언어 모델과 동일함. 입력 문장의 15%를 마스크 처리해 마스크 처리된 토큰이 원래 어떤 토큰이었는지를 맞히며 학습함.
- 판별자 모델은 생성자 모델이 복원한 결과값을 입력으로 받음 → 각 토큰이 원본 토큰인지 바뀐 토큰인지를 판별함.

- 생성적 적대 신경망(GAN)과 유사하지만 몇 가지 차이점이 있음.
 - ELECTRA는 생성 모델이 정확히 예측한 토큰을 원본 토큰으로 인식
 - GAN은 생성 모델과 판별 모델을 적대적으로 학습함. 생성 모델은 판별 모델이 실제 데이터와 구분하기 어렵게 학습하고, 판별 모델은 실제 데이터와 생성된 데이터를 더욱 잘 구분하게 학습함.

T5

- 인코더-디코더 모델 구조를 바탕으로 입출력을 모두 토큰 시퀀스로 처리하는 텍스트-텍스트 구조
- 텍스트-텍스트 구조로 입출력의 형태가 자유롭고 모델 구조상 유연성과 확장성이 뛰어남.
- 다양한 자연어 처리 작업에서 높은 성능을 보이는 모델



- 입력과 출력이 모두 토큰(텍스트) 시퀀스이기 때문에 입력과 출력 간의 관계를 더욱 세밀하게 다룰 수 있다.
- T5는 사전 학습 후 미세 조정 단계에서 해당 작업의 데이터를 이용해 모델을 조정해 최적의 성능을 얻을 수 있다.
- CoLA 데이터셋: 문법적으로 허용 가능한 문장과 그렇지 않은 문장 구분 가능

- STS-B 데이터셋: 문장 간 의미적 유사성 측정 가능
- C4 데이터셋: 다양한 자연어 처리 작업을 수행할 수 있게 사전 학습됨

사전 학습 방식

- 비지도 학습 방식으로 입력 문장의 일부 구간을 마스킹해 입력 시퀀스를 처리
- 출력 시퀀스는 실제 마스킹된 토큰과 마스크 토큰의 연결로 구성됨.
- 센티넬 토큰: 문장마다 유일한 마스크 토큰을 의미함. <extra_id_n>을 예측하는 것이 목적임.
- 미세 조정은 지도 학습 방식으로 학습됨.
- 번역, 언어 수용성, 문장 유사도, 문서 요약 등 다양한 작업에 활용할 수 있음.
- T5 모델의 입력과 출력은 텍스트 시퀀스 토큰들로 이뤄져 있으며, 인코더-디코더 T5 모델에 입력됨.